

Computing and Visualizing Prime Numbers

Austin Burt

Abstract

This implements a parallel segmented sieve of Eratosthenes in C to generate primes efficiently, benchmark process-level and thread-level parallelism, empirically verify the Prime Number Theorem, and visualize prime distributions through the Ulam spiral. Our parallel implementation achieves about a 380% speedup with 64 workers before overhead dominates. The empirical prime density matches the theoretical prediction $1/\ln(x)$ across ranges up to 8.0×10^9 (Took the sieve around two and half minutes to compute)

1 Introduction

Prime numbers (integers greater than 1 divisible only by 1 and themselves) are the fundamental building blocks of the integers. Their distribution has fascinated mathematicians for millennia, from Euclid’s proof of their infinitude to Riemann’s still-unproven hypothesis about the zeros of the zeta function.

This paper explores three interconnected aspects of prime numbers: efficient computation via sieving algorithms, empirical verification of the Prime Number Theorem, and visual exploration of hidden structure through the Ulam spiral. It implements a parallel segmented sieve in C¹, benchmark its scaling across multiple processes and threads, and generate visualizations that confirm theoretical predictions about prime density and gap distributions.

2 The Sieve of Eratosthenes

The sieve of Eratosthenes is one of the oldest algorithms for finding all primes up to a given limit n . The algorithm proceeds as follows:

1. Write the integers $2, 3, 4, \dots, n$.
2. Let $p = 2$, the smallest unmarked number, it is prime.
3. Cross off all multiples of p greater than p .

¹Full source code and benchmarks available at <https://github.com/idoneve/Prime-Plotter>.

4. Find the next unmarked number and set p to it. Repeat from step 3.
5. Stop when $p > \sqrt{n}$.

After the algorithm terminates, all unmarked numbers are prime. The only optimization needed is that once $p > \sqrt{n}$, any remaining composite must have a prime factor $\leq \sqrt{n}$, so all numbers have already been crossed off.

Figure 1 visualizes the sieve at each prime step for $n = 100$. Black squares represent unmarked (candidate prime) numbers, while gray squares have been eliminated as composite.

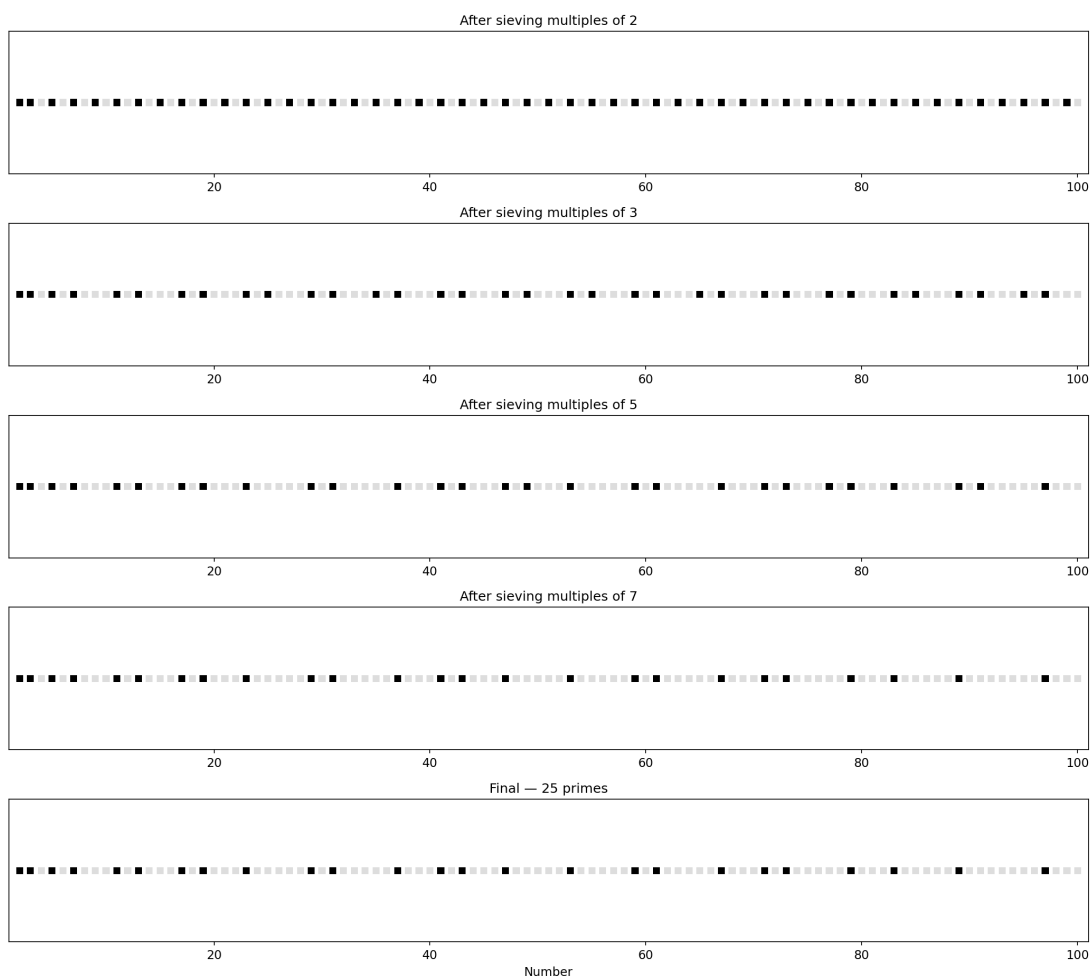


Figure 1: The sieve of Eratosthenes for $n = 100$, shown after sieving multiples of each prime $p \leq \sqrt{100}$. The final row shows all 25 primes ≤ 100 .

The complexity of this algorithm is $O(n \log \log n)$, making it nearly linear. For the computations, it uses a *segmented* variant that sieves the range in blocks, reducing memory usage and enabling parallelization. We delegate the actual sieving to the highly-optimized `primesieve` library [1].

3 Parallel Implementation

Generating primes up to 8.0×10^9 requires sieving eight billion numbers, a task well-suited to parallel computation. The C implementation distributes work across multiple *processes* (via `fork()`) and multiple *threads* per process (via `pthread`s).

3.1 Design

The number line is divided into equal segments. Each process is assigned a contiguous range, and within each process, threads subdivide that range further. Each thread independently sieves its segment using `primesieve` and writes results to a separate CSV file. Because segments are disjoint, no synchronization or communication between workers is needed, so this is an *embarrassingly parallel* workload.

3.2 Processes vs. Threads

Processes and threads represent two fundamental approaches to parallelism:

- **Processes** have separate memory spaces. They avoid cache contention and lock overhead, but have higher creation cost.
- **Threads** share memory within a process. They are cheaper to create, but compete for caches and memory bandwidth, limiting scaling.

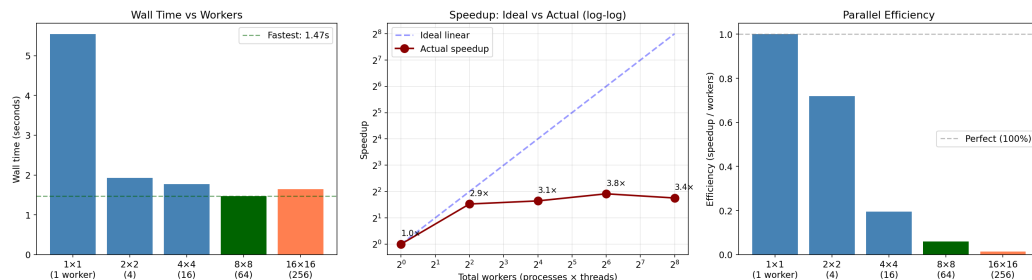


Figure 2: Parallel scaling benchmarks.

(Left) Wall time decreases from 5.5s (1 worker) to 1.5s (64 workers), then increases to 1.6s at 256 workers as process creation overhead dominates.

(Center) Speedup (log-log scale). Ideal linear scaling is shown dashed; actual speedup plateaus at 3.8x.

(Right) Parallel efficiency drops from 72% at 4 workers to below 20% at 64 as overhead grows.

Figure 2 summarizes the benchmark results. The sweet spot occurs at 64 workers (8 processes $\times$ 8 threads), achieving a 3.8 times speedup. Beyond this, each worker's

segment becomes too small, at 256 workers, each chunk is only $\sim 39,000$ numbers, and the overhead of spawning processes and threads outweighs the benefit of further subdivision.

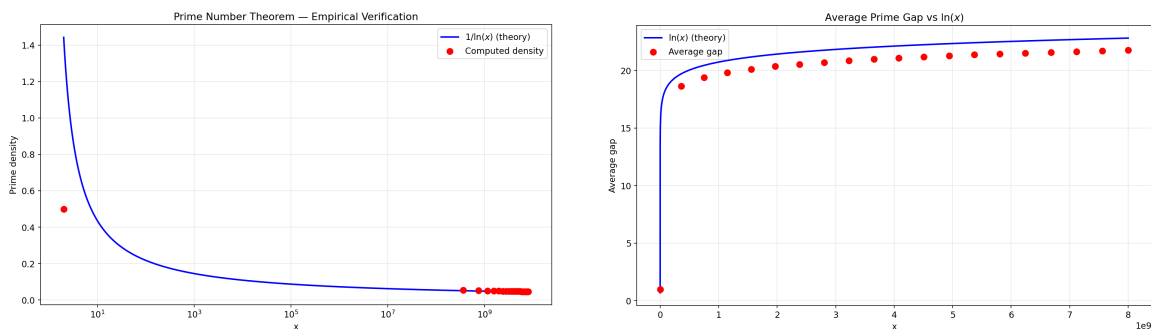
4 The Prime Number Theorem

Theorem 4.1 (Prime Number Theorem). *Let $\pi(x)$ denote the number of primes $\leq x$. Then*

$$\pi(x) \sim \frac{x}{\ln x}$$

as $x \rightarrow \infty$. Equivalently, the density of primes near x is approximately $1/\ln(x)$, and the average gap between consecutive primes near x is approximately $\ln(x)$.

Using the parallel sieve, we computed primes up to 8.0×10^9 and verified the PNT empirically. Figure 3 shows the results.



(a) Prime density vs. x . Red dots are computed densities; the blue curve is $1/\ln(x)$.

(b) Average prime gap vs. x . Red dots are computed averages; the blue curve is $\ln(x)$.

Figure 3: Empirical verification of the Prime Number Theorem. Both density and average gap closely follow theoretical predictions.

The close agreement between computed data (red points) and theoretical curves (blue lines) confirms the PNT across the range 1 to 8.0×10^9 . The density decays from approximately 5.2% near 2.4×10^8 to 4.6% near 2.6×10^9 , matching the logarithmic decay predicted by theory.

5 Prime Gaps

The difference $g_n = p_{n+1} - p_n$ between consecutive primes is called a prime gap. Figure 4 shows the distribution and behavior of prime gaps in the computed range. In which twin primes constitute approximately 0.28–0.36% of gaps, with the fraction decreasing as predicted. The maximum observed gap was around 350, consistent with Cramér’s conjecture that maximal gaps grow like $\ln^2(p_n)$.

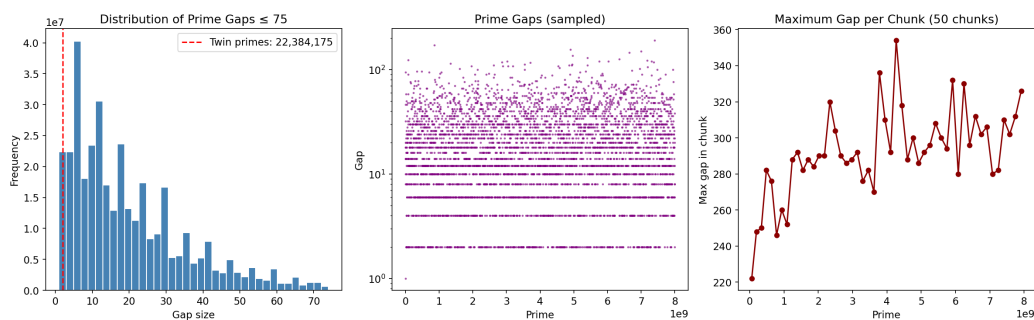


Figure 4: Prime gap analysis.

(Left) Distribution of gaps ≤ 75 , the first bar shows twin primes.
 (Center) Scatter plot of individual gaps, with logarithmic y-axis.
 (Right) Maximum gap per chunk, showing slow growth. With the max gap around 350.

6 The Ulam Spiral

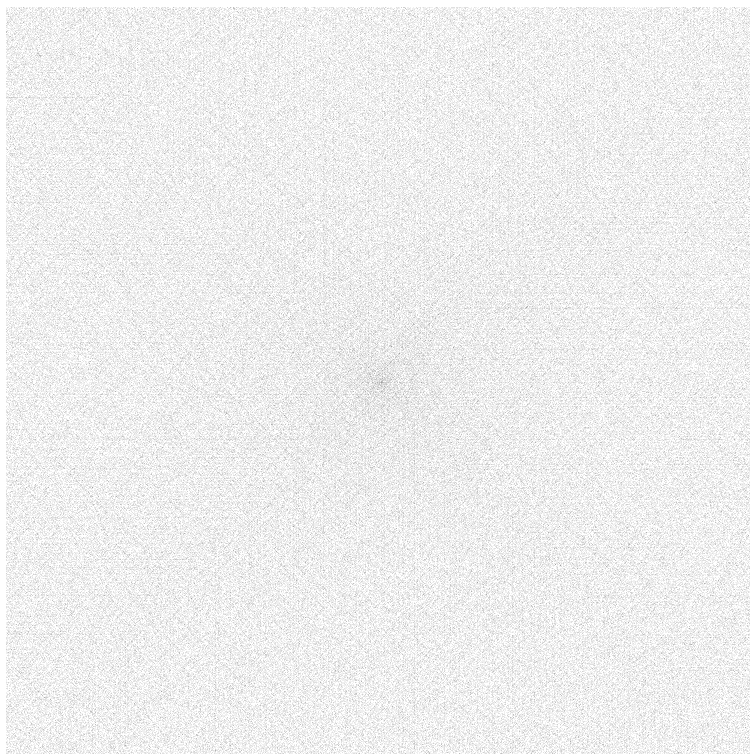


Figure 5: Ulam spiral of primes up to 8.0×10^9 (Feel free to zoom in as much as you want). Black dots are primes; diagonal striations correspond to prime-generating quadratic polynomials such as Euler's $n^2 + n + 41$.

Remark 6.1. In 1963, Stanisław Ulam discovered a striking visual pattern by arranging numbers in a square spiral and marking the primes. Diagonal lines emerge, these correspond to quadratic polynomials that produce unusually many primes. The most famous diagonal corresponds to Euler’s polynomial $n^2 + n + 41$, which produces primes for $n = 0, 1, \dots, 39$. Other diagonals correspond to polynomials like $4n^2 + 2n + 41$ and $4n^2 + 4n + 59$. The existence of such prime-rich quadratics is connected to class numbers of imaginary quadratic fields, but a complete explanation remains an open problem in number theory.

Remark 6.2. The Ulam spiral demonstrates that prime numbers, despite their apparent randomness, possess subtle structure. It is a powerful reminder that visualization can reveal patterns invisible to purely analytic approaches.

7 Conclusion

This project demonstrates the interplay between theory and computation in number theory. The parallel segmented sieve efficiently generates primes up to 8.0×10^9 , enabling empirical verification of the Prime Number Theorem and exploration of prime gaps. The Ulam spiral visualization reveals hidden quadratic structure in prime distributions.

Key findings include: (1) parallel speedup reaches $3.8\times$ at 64 workers before overhead dominates, (2) empirical prime density matches $1/\ln(x)$ with high precision, and (3) diagonal patterns in the Ulam spiral persist at large scales, reflecting deep connections between primes and quadratic polynomials.

References

- [1] K. Walisch, *primesieve: Fast prime number generator*, <https://github.com/kimwalisch/primesieve>.
- [2] Y. Zhang, *Bounded gaps between primes*, *Annals of Mathematics*, 179(3):1121–1174, 2014.
- [3] K.-D. Crisman, *Number Theory: In Context and Interactive*, Gordon College, 2021, <https://math.gordon.edu/ntic/>.